This formal definition is the mathematical heart of **Region Abstraction**. It explains how to take an infinite number of possible clock values and group them into a finite number of "equivalent" shapes.

The goal is to say: "Even though v and v' are different exact times, the automaton will behave **exactly the same way** in the future for both."

Here is the breakdown of the three conditions that must be met for two time-points to be equivalent:

---

1. The "Out of Bounds" Condition (i)

$$v(x) > c_m \text{ and } v'(x) > c_m$$

c_m is the largest constant any clock is compared to in the guards (e.g., if you only have `x < 5`, then $c_m = 5$).

- **The Logic:** If both clocks have passed the maximum threshold, the exact value doesn't matter anymore. Whether $x = 100$ or $x = 1000$, a guard like `x < 5` will always be false. They are now "equivalently large."

---

2. The "Integer Part" Condition (iia)

$$\lfloor v(x) \rfloor = \lfloor v'(x) \rfloor \text{ and } fr(v(x)) = 0 \Leftrightarrow fr(v'(x)) = 0$$

- $\lfloor v(x) \rfloor$**:** This is the integer part (floor). If $v(x) = 2.3$ and $v'(x) = 2.7$, they both have an integer part of 2.
- $fr(v(x)) = 0$**:** This checks if the clock is currently a **whole integer** (like exactly 2.0).
- **The Logic:** To be equivalent, two valuations must be in the same "integer open interval" (e.g., between 2 and 3) OR they must both be exactly the same integer. If one is 2.0 and the other is 2.1, they are NOT equivalent because one might trigger a `x == 2` guard and the other won't.

---

3. The "Fractional Order" Condition (iib)

$$fr(v(x)) \sim fr(v(y)) \Leftrightarrow fr(v'(x)) \sim fr(v'(y))$$

This is the most important part for multiple clocks (x and y). It looks at the **ordering of the decimal parts**.

- **The Logic:** It asks: "Which clock will hit the next integer first?"
- **Example:** * Let $v = (x : 1.2, y : 1.5)$. Here, $fr(x) < fr(y)$. This means y will hit the next integer (2.0) before x does.
    - Let $v' = (x : 1.1, y : 1.8)$. Here, $fr(x) < fr(y)$ still holds.
    - **Result:** $v \cong v'$. Even though the numbers are different, the *sequence* of future events is identical.

If we had $v'' = (x : 1.5, y : 1.2)$, then x would hit the next integer first. This is a **different region** because the system might have a transition that only works if x hits an integer while y is still small.

Summary: Why do we do this?

Without this relation, a Timed Automaton has an **infinite state space** (because time is continuous: 1.1, 1.11, 1.111...).

By applying these rules, we partition the 2D plane (for two clocks) into:

1. **Points** (where both clocks are integers).
2. **Lines** (where one clock is an integer).
3. **Triangles** (where no clocks are integers, and we know their fractional order).

This turns the infinite problem into a **finite graph problem**, allowing tools like TARZAN or UPPAAL to actually finish their calculations!